

GROUP 04 · CAPSTONE PROJECT

Property & Home Insurance AI Copilot

Damage Assessment · Repair Estimation · Claim Filing · Contractor Network

Domain	Property & Home Insurance
AI Pattern	Agentic RAG + Persistent Conversational Memory + Multi-Tool Dispatch
Stack Hint	LangChain · FastAPI · FAISS · Streamlit · Docker · AWS
Duration	Day 19 & 20 — Full Capstone
Team Size	4–5 Participants
Type	Problem Statement Architecture Reference Evaluation Rubric

Enterprise AI Training Program · 15-Day Bootcamp Series

1. Project Overview

Group 04 is tasked with building a production-grade **Property & Home Insurance AI Copilot** — an intelligent assistant that supports homeowners and claims handlers through the full property insurance lifecycle: policy coverage Q&A, damage assessment guidance, repair cost estimation, claim filing assistance, contractor network lookup, and claim status tracking.

The key technical differentiator for this project is **Agentic RAG with Persistent Conversational Memory**. Unlike a stateless Q&A bot, this system remembers what the user has shared across turns — property details, damage descriptions, claim IDs, and coverage type — and uses that accumulated context to provide progressively richer, more personalised responses without asking the user to repeat themselves. A multi-tool dispatch layer routes queries to the appropriate specialised tool based on intent.

Domain	Property & Home Insurance
AI Pattern	Agentic RAG + Persistent Memory + Multi-Tool Dispatch
User Personas	Homeowners · Claims Handlers · Surveyors · Repair Contractors
Core Challenge	Memory-augmented agent that accumulates property and damage context across turns and dispatches to 4 specialised tools
Final Deliverable	Live URL (AWS) + Streamlit UI + FastAPI + FAISS + Memory + Evaluation Scorecard
Duration	Day 19 & 20 (Full capstone + deployment demo)
Team Size	4–5 participants

2. Problem Statement

When a home is damaged — by fire, flood, storm, theft, or structural failure — the policyholder enters a stressful, document-heavy claims process. They must understand what their policy covers, document the damage correctly, get repair estimates, find approved contractors, file the claim accurately, and track its progress. Each of these steps currently requires separate interactions with different systems and agents.

The compounding problem is memory loss across interactions. A homeowner who called yesterday to describe storm damage to their roof should not have to repeat those details when they call back to ask about approved contractors. Claims handlers who gathered damage information in one session should be able to reference it in the next. The current stateless, siloed experience adds friction at exactly the moment the customer needs speed and clarity.

■ Your Mission

Build a memory-augmented AI copilot that accumulates property context, damage descriptions, and claim details across conversation turns, and uses that persistent memory to answer follow-up questions without repetition — routing to the correct specialised tool (policy RAG, damage estimator, contractor lookup, claim tracker) based on query intent.

The system must demonstrate that a user can describe their property and damage in turn 1, ask for repair cost estimates in turn 3, and request approved contractors in turn 5 — with the agent correctly using the property type and damage type captured earlier across all three turns.

2.1 Key Pain Points Your System Must Address

- **Coverage Ambiguity:** Homeowners struggle to understand whether a specific peril (e.g., subsidence, gradual water damage, mould) is covered under their policy or excluded.
- **Damage Documentation Guidance:** After an incident, policyholders are unsure what photos, reports, and evidence are required to support their claim.
- **Repair Cost Opacity:** Neither the homeowner nor the claims handler has quick access to indicative repair cost ranges for common damage types.
- **Contractor Network Friction:** Policyholders must call or browse to find insurer-approved contractors — a slow and error-prone process.
- **Memory-less Interactions:** Every call or chat session starts from zero — the agent has no recollection of prior damage descriptions, property details, or claim context.
- **Claim Status Uncertainty:** Policyholders cannot easily get a current status update on a submitted claim without calling a helpline.
- **Underinsurance Risk:** Many homeowners are unaware their declared sum insured may not reflect current rebuilding costs — the agent should surface this when relevant.

■ ■ Non-Negotiable Safety Constraint

The copilot must NEVER provide a final claims decision, approve or reject a claim, or provide a binding repair cost quote. All cost figures must be clearly labelled as indicative estimates. The system must not fabricate contractor names, addresses, or availability data. Every policy-grounded answer must cite source document and page.

3. Provided Dataset & Knowledge Base

The following synthetic documents and structured datasets are provided. All data is fictional and created for training purposes only.

#	Document / Dataset	Format	Size	Primary Use Case
1	Home Insurance Policy Document (Standard / Comprehensive / Landlord tiers)	PDF	68 pp	Coverage terms, perils covered, exclusions, sums insured
2	Policy Summary & Key Facts Document (All three tiers)	PDF	6 pp × 3	Quick-reference coverage limits, excess amounts, add-ons
3	Claims Procedure & Evidence Guide	PDF	14 pp	Step-by-step claims process, required documentation, timelines
4	Property Damage Repair Cost Table (Damage type × property category)	CSV	280 rows	Indicative repair cost ranges by damage type and property size
5	Approved Contractor Network Directory (Postcode × trade × insurer tier)	CSV	420 rows	Contractor name, trade type, coverage area, contact, tier approval
6	Perils & Exclusions Reference Guide	PDF	20 pp	What is and is not covered per peril: flood, fire, storm, theft, subsidence
7	Contents & Valuables Coverage Guide	PDF	10 pp	High-value items, limits, scheduled items, away-from-home cover
8	Rebuilding Cost Index by Property Type	CSV	180 rows	Current rebuild cost benchmarks by property type, size, region
9	Accidental Damage & Legal Expenses Add-On Guide	PDF	8 pp	Optional add-on coverage details, limits, exclusions

#	Document / Dataset	Form at	Size	Primary Use Case
10	Home Insurance Glossary	PDF	11 pp	Definitions: excess, indemnity, new-for-old, reinstatement, subrogation, etc.

■ Memory Design Hint

The persistent memory store should accumulate structured property context as the conversation progresses: property_type, property_size, damage_type, peril_category, policy_tier, claim_id, and postcode. Each tool call should read from and write back to this memory. A user who mentions 'my 3-bed semi in Manchester' in turn 1 should not need to repeat this when asking for contractors in turn 4.

4. Expected System Architecture & Data Flow

This project introduces **Persistent Conversational Memory** as the core architectural differentiator. Study both diagrams: Figure 1 shows the 5-layer system architecture, and Figure 2 shows the detailed Agentic RAG + Memory + Tool-Use internal architecture your team must implement.

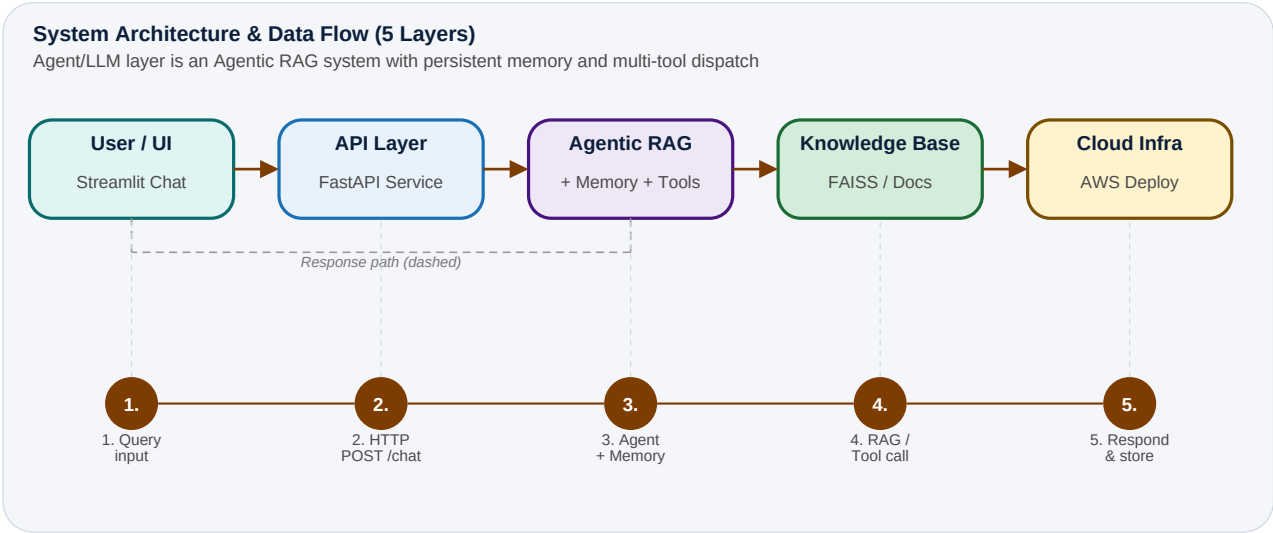


Figure 1: Five-layer system architecture — Agent layer implements Agentic RAG with memory

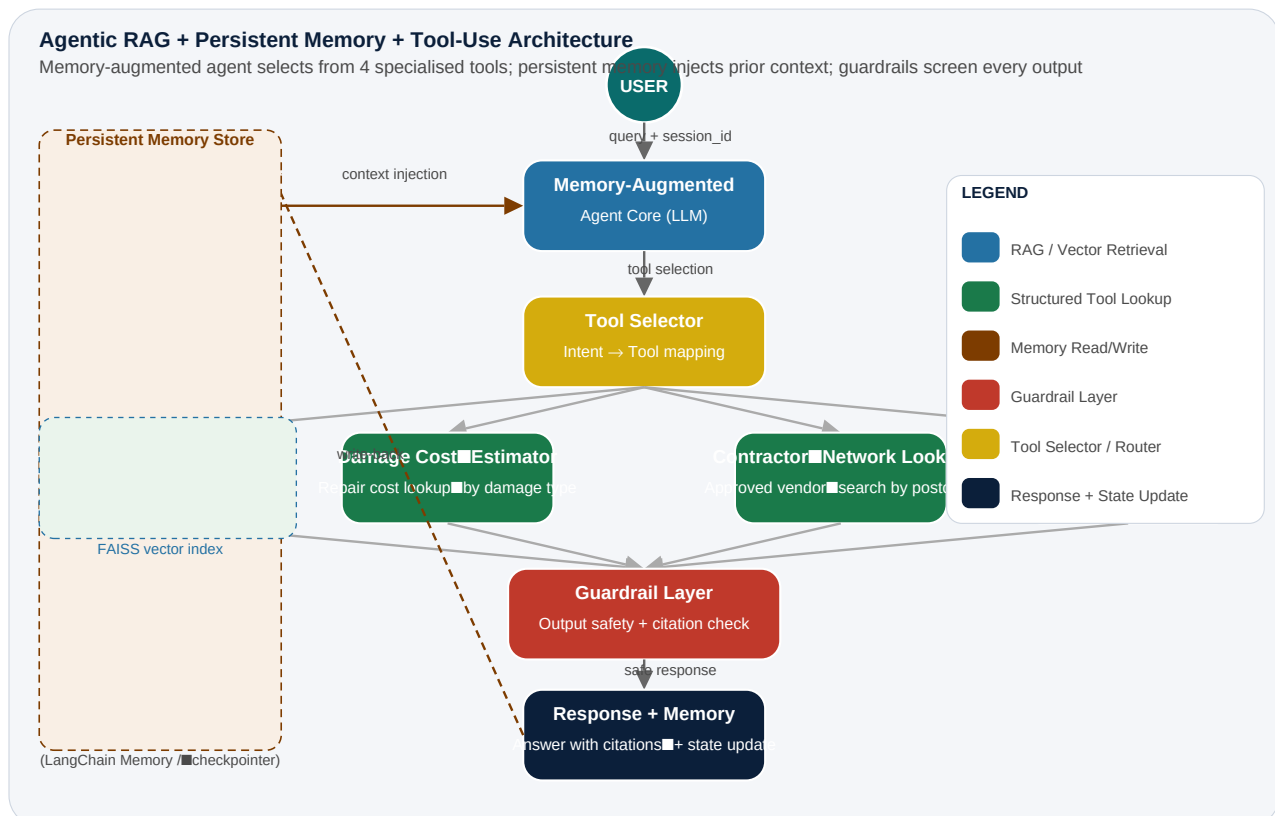


Figure 2: Agentic RAG + Persistent Memory architecture — memory injects prior property context into every agent call; tool selector dispatches to 4 specialised tools; memory is updated after every response

4.1 Component Responsibilities

Component	Type	Responsibility
Persistent Memory Store	Memory Layer	Stores and retrieves structured property context (property type, damage type, postcode, policy tier, claim ID) and full conversation history. Injected into every agent call to provide continuity without repetition.
Memory-Augmented Agent	LLM Core	Receives the user query enriched with memory context. Determines intent and delegates to the Tool Selector. Synthesises the final response using tool output + memory.
Tool Selector	Router Node	Maps query intent to the correct tool: policy coverage questions → RAG Retriever; damage cost queries → Damage Estimator; contractor queries → Contractor Lookup; claim status queries → Claim Tracker.
Policy RAG Retriever	RAG Tool	Semantic search over FAISS index of all policy PDFs. Returns top-k chunks with source citation, filtered by policy tier metadata. Used for coverage, exclusion, and procedure questions.
Damage Cost Estimator	Structured Tool	Queries the repair cost CSV filtered by damage type and property category. Returns indicative cost range. Reads property context from memory to avoid asking for details already provided.
Contractor Network Lookup	Structured Tool	Queries the contractor directory CSV filtered by postcode (from memory), trade type (from damage type in memory), and policy tier. Returns a shortlist of approved contractors.
Claim Status Tracker	Structured Tool	Simulates a claim lookup by claim ID. Returns current status, next steps, and any pending actions. Claim ID is read from memory if provided in a prior turn.

Component	Type	Responsibility
Guardrail Layer	Safety Filter	Screens every output for: fabricated contractor details, binding cost commitments, claim decisions, PHI/PII exposure, and prompt injection. Adds mandatory citations and disclaimers.

4.2 Required Data Flow (5 Steps)

- **Step 1 — Query Input:** User describes property details or asks a question. Session ID passed with every request.
- **Step 2 — HTTP POST /chat:** FastAPI receives query + session_id + optional policy_tier. Validates and forwards to the agent.
- **Step 3 — Agent + Memory:** Prior property context and conversation history are loaded from the memory store and injected into the agent prompt. The agent interprets the query in full context.
- **Step 4 — RAG / Tool Call:** Tool Selector routes to the correct tool. RAG retriever performs FAISS search; structured tools query CSV datasets using property context from memory.
- **Step 5 — Respond & Store:** LLM synthesises a cited response, guardrails screen it, and the updated property context is written back to the memory store for future turns.

5. Functional Requirements

Your system must fulfil the following capabilities. Implementation decisions are yours.

#	Requirement	Acceptance Condition
FR-01	Coverage & Perils Q&A;	User asks whether a specific peril or damage type is covered → system returns accurate answer with policy tier distinction and source citation.
FR-02	Persistent Memory	Property context shared in turn N must be available and used in turns N+2 and beyond without the user re-entering it. Demonstrable during live demo across minimum 4 turns.
FR-03	Damage Cost Estimation	Given damage type and property category (from query or memory), system returns indicative repair cost range from the reference table with clear 'indicative estimate' labelling.
FR-04	Contractor Network Lookup	Given postcode and damage type (from query or memory), system returns a shortlist of approved contractors from the directory with trade type and contact details.
FR-05	Claim Filing Guidance	System walks user through the claims process step-by-step, listing required documentation and filing deadlines from the claims procedure guide.
FR-06	Claim Status Lookup	Given a claim ID (from query or memory), system returns current claim status and pending actions.
FR-07	Contents & Add-On Queries	System answers questions about contents coverage limits, high-value items, and optional add-ons (accidental damage, legal expenses) with citations.
FR-08	Underinsurance Flag	When sum insured is discussed, system proactively references the rebuilding cost index and flags potential underinsurance risk if applicable.
FR-09	Policy Tier Awareness	Responses must correctly reflect coverage differences between Standard, Comprehensive, and Landlord policy tiers.
FR-10	Guardrails	System must block: binding cost quotes, claim approval/rejection, fabricated contractor data, PHI/PII in output, and prompt injection attempts.

5.1 Non-Functional Requirements

- **Performance:** API must respond within 4 seconds at P95 including memory read/write overhead.
- **Memory Durability:** Property context must survive API restarts within a demo session window.
- **Memory Transparency:** The UI must show a summary of what is currently in memory (property type, damage type, postcode, policy tier) so the evaluator can verify persistence.
- **Security:** All credentials stored as environment variables. No hardcoded secrets.
- **Portability:** Full application runs via a single Docker command.

6. Evaluation Criteria & Scoring Rubric

Total: 100 points + bonus. Automated scoring via LangSmith + DeepEval combined with live demo assessment.

Dimension	Weight	What Will Be Assessed	Minimum Threshold
RAG Accuracy	20 pts	Faithfulness and Answer Relevancy on 30-question test set across all tool types.	Faithfulness ≥ 0.85 Relevancy ≥ 0.80
Memory Persistence	25 pts	Evaluator runs a scripted 5-turn conversation. Turns 3, 4, and 5 must correctly use context from turns 1 and 2 without re-prompting. Each memory-correct turn = ~8 pts.	3 of 5 turns must use prior context
System Completeness	20 pts	All 10 FRs (FR-01 to FR-10) demonstrated live. Each = 2 pts.	8 of 10 FRs must pass
Guardrail Effectiveness	15 pts	3 adversarial inputs: binding cost quote request, claim approval request, fabricated contractor name injection. Each blocked = 5 pts.	All 3 blocked
Deployment & Ops	10 pts	Live AWS URL, Docker confirmed, LangSmith traces show memory read/write per turn, secrets via env vars.	Public URL live during demo
Presentation	10 pts	Memory architecture explained, live memory state panel shown, tool dispatch demonstrated across 4 tools.	Must match implementation

6.1 Test Dataset Requirements (30 Q&A; Pairs)

Coverage & Perils Q&A; (RAG)	10 questions	All 3 policy tiers; mix of covered and excluded perils
Damage Cost Estimation	6 questions	Different damage types and property categories
Contractor Lookup	5 questions	Different postcodes, trade types, and policy tiers
Claim Filing & Status	5 questions	Procedure steps, required docs, status by claim ID
Contents & Add-On Queries	4 questions	High-value items, accidental damage, legal expenses

■ Memory-Specific Evaluation Requirement

In addition to standard DeepEval and LangSmith scoring, the evaluator will run a scripted 5-turn memory test: Turn 1 — user provides property type, size, and damage. Turn 2 — unrelated policy question. Turn 3 — repair cost query (must use property context from turn 1). Turn 4 — contractor query (must use postcode from turn 1). Turn 5 — coverage question (must reference damage type from turn 1). Each of turns 3, 4, 5 that correctly uses prior context without re-prompting scores full marks.

6.2 Creativity & Innovation Bonus (+15 pts)

Teams are encouraged to go beyond the base requirements:

- **+5 pts — Photo Damage Assessment:** Accept a photo of property damage as input; use a vision model to identify the damage type and pre-populate the memory context automatically.
- **+5 pts — Underinsurance Calculator:** Build an interactive flow where the user provides property details and the system compares their declared sum insured against the rebuilding cost index, flagging any gap with a percentage.
- **+5 pts — Proactive Memory Prompting:** Agent proactively asks targeted follow-up questions to fill gaps in memory context (e.g., 'You mentioned storm damage — could you confirm your property type so I can look up contractors?').

7. Deliverables & Demo Checklist

Items marked **CRITICAL** are required for a passing grade.

CRITICAL	Memory demonstrated: property context from turn 1 correctly used in turn 3+ without re-prompting (minimum 3 consecutive memory-correct turns)
CRITICAL	All 4 tools demonstrated: RAG retriever, damage estimator, contractor lookup, and claim tracker each invoked at least once
CRITICAL	UI memory panel: current memory state (property type, damage type, postcode, policy tier) visible in the Streamlit UI
CRITICAL	Policy Q&A; returns source citation: document name, section, and page number present in response
CRITICAL	Application live on public AWS URL during the demo window
REQUIRED	Damage cost estimate returned with 'indicative estimate' label and correct property/damage filter from memory
REQUIRED	Contractor lookup returns filtered shortlist using postcode from memory
REQUIRED	LangSmith traces show memory read and write operations per turn
REQUIRED	DeepEval scorecard: Faithfulness and Answer Relevancy on 30-question test set
REQUIRED	Guardrails block binding cost quote and claim approval requests
REQUIRED	Docker confirmed: build and run demonstrated or evidenced
BONUS	Photo damage assessment, underinsurance calculator, or proactive memory prompting (+5 pts each)

8. Suggested Timeline (Day 19 & 20)

All phases must be complete before the demo window on Day 20. Phase 2 (Memory Design) is the most critical — design the memory schema before building tools.

Phase	Focus Area	Time Box	Exit Condition
Phase 1	Data ingestion — PDF loading, CSV parsing, FAISS indexing with policy tier + damage type metadata	1.5 hrs	FAISS index built; CSV datasets queryable with filters

Phase	Focus Area	Time Box	Exit Condition
Phase 2	Memory design — define memory schema, implement read/write, test context injection across 3 turns	2.0 hrs	Agent correctly uses context from turn 1 in turn 3 for at least 2 different context types
Phase 3	Tool implementation — 4 tools built and unit tested (RAG, cost estimator, contractor lookup, claim tracker)	1.5 hrs	Each tool returns correct output for 2 test inputs
Phase 4	API + UI — FastAPI /chat endpoint, Streamlit UI with memory state panel	1.0 hrs	UI connected, memory panel visible, tool dispatch working
Phase 5	Guardrails + adversarial testing	0.5 hrs	All 3 guardrail categories blocked
Phase 6	Evaluation — 30-question test set, 5-turn memory test, DeepEval, LangSmith	1.0 hrs	Scorecard produced, Faithfulness $\geq$ 0.85, memory test passes 3/5 turns
Phase 7	Docker build, AWS deploy, URL confirmed, demo rehearsal	1.0 hrs	Live URL accessible, demo rehearsed once including scripted memory test

■ Final Reminder

This is a problem statement, not a solution guide. Design the memory schema first — everything else depends on it. A system with perfect RAG but broken memory will fail the most important evaluation dimension.

Good luck, Group 04. Build an agent that remembers, reasons, and responds — never asking the user to repeat themselves.